

Apache Hadoop and Spark: Introduction and Use Cases for Data Analysis

Afzal Godil

Information Access Division, ITL, NIST

Outline

- Growth of big datasets
- Introduction to Apache Hadoop and Spark for developing applications
- Components of Hadoop, HDFS, MapReduce and HBase
- Capabilities of Spark and the differences from a typical MapReduce solution
- Some Spark use cases for data analysis

Growth of Big Datasets

- Internet/Online Data
 - Clicks
 - Searches
 - Server requests
 - Web logs
 - Cell phone logs
 - Mobile GPS locations
 - User generated content
 - Entertainment (YouTube, Netflix, Spotify, ...)
- Healthcare and Scientific Computations
 - Genomics, medical images, healthcare data, billing data
- Graph data
 - Telecommunications network
 - Social networks (Facebook, Twitter, LinkedIn, ...)
 - Computer networks
- Internet of Things
 - RFID
 - Sensors
- Financial data

Data

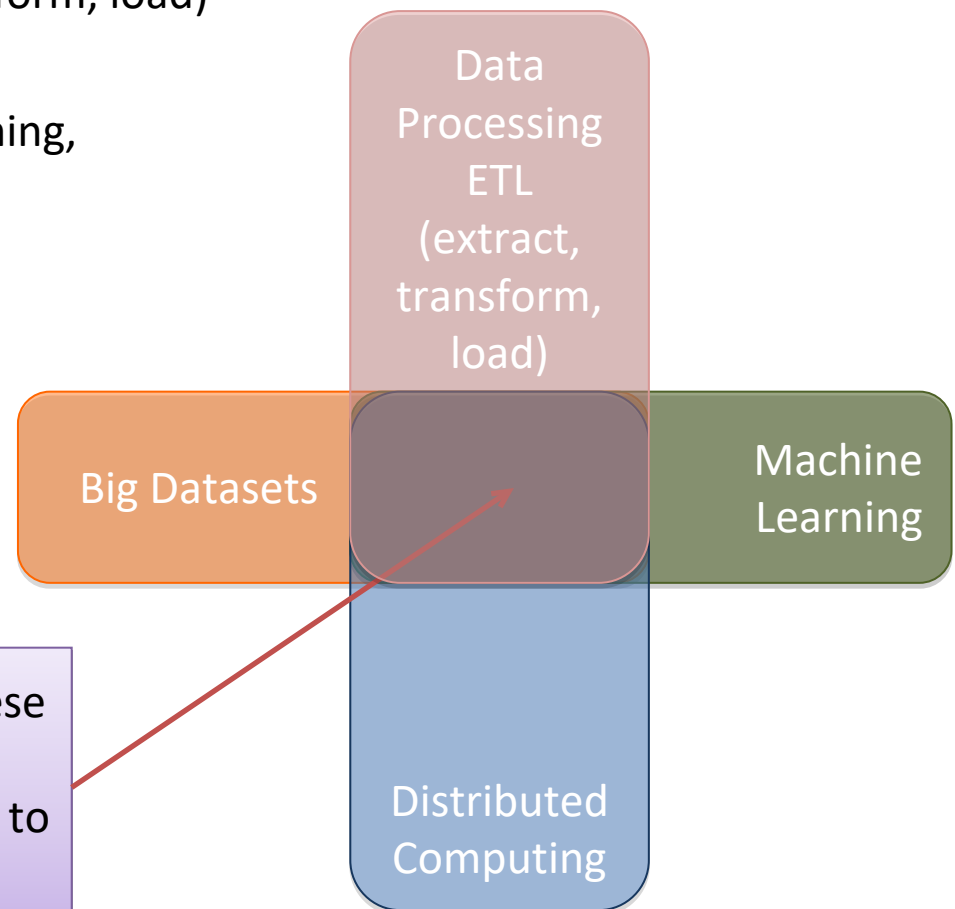
- The Large Hadron Collider produces about 30 petabytes of data per year
- Facebook's data is growing at 8 petabytes per month
- The New York stock exchange generates about 4 terabyte of data per day
- YouTube had around 80 petabytes of storage in 2012
- Internet Archive stores around 19 petabytes of data

Cloud and Distributed Computing

- The second trend is pervasiveness of cloud based storage and computational resources
 - For processing of these big datasets
- Cloud characteristics
 - Provide a scalable standard environment
 - On-demand computing
 - Pay as you need
 - Dynamically scalable
 - Cheaper

Data Processing and Machine learning Methods

- Data processing (third trend)
 - Traditional ETL (extract, transform, load)
 - Data Stores (HBase,)
 - Tools for processing of streaming, multimedia & batch data
- Machine Learning (fourth trend)
 - Classification
 - Regression
 - Clustering
 - Collaborative filtering

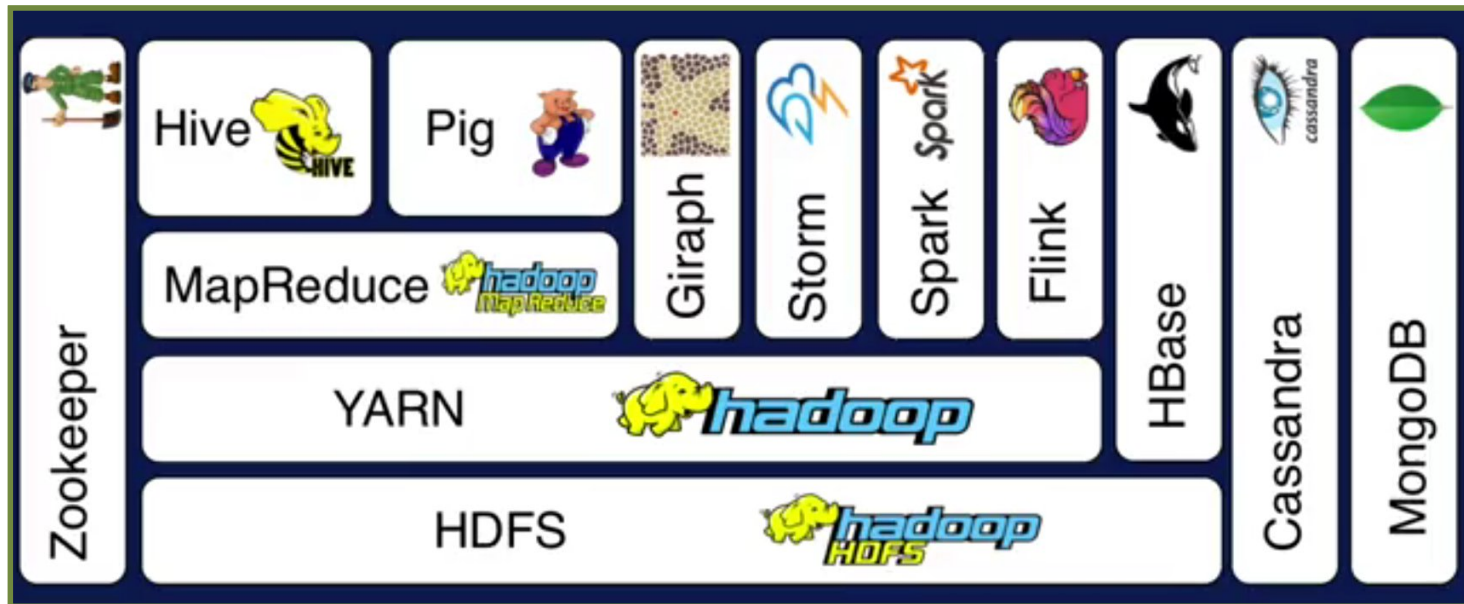
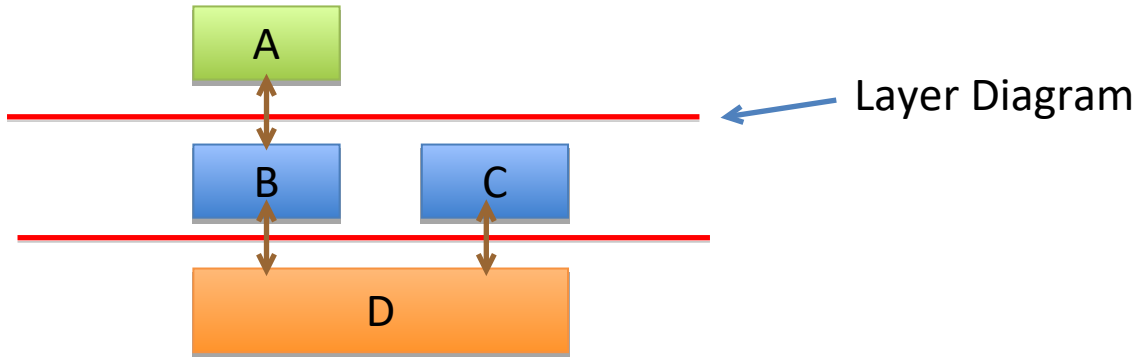


Working at the Intersection of these four trends is very exciting and challenging and require new ways to store and process **Big Data**

Hadoop Ecosystem

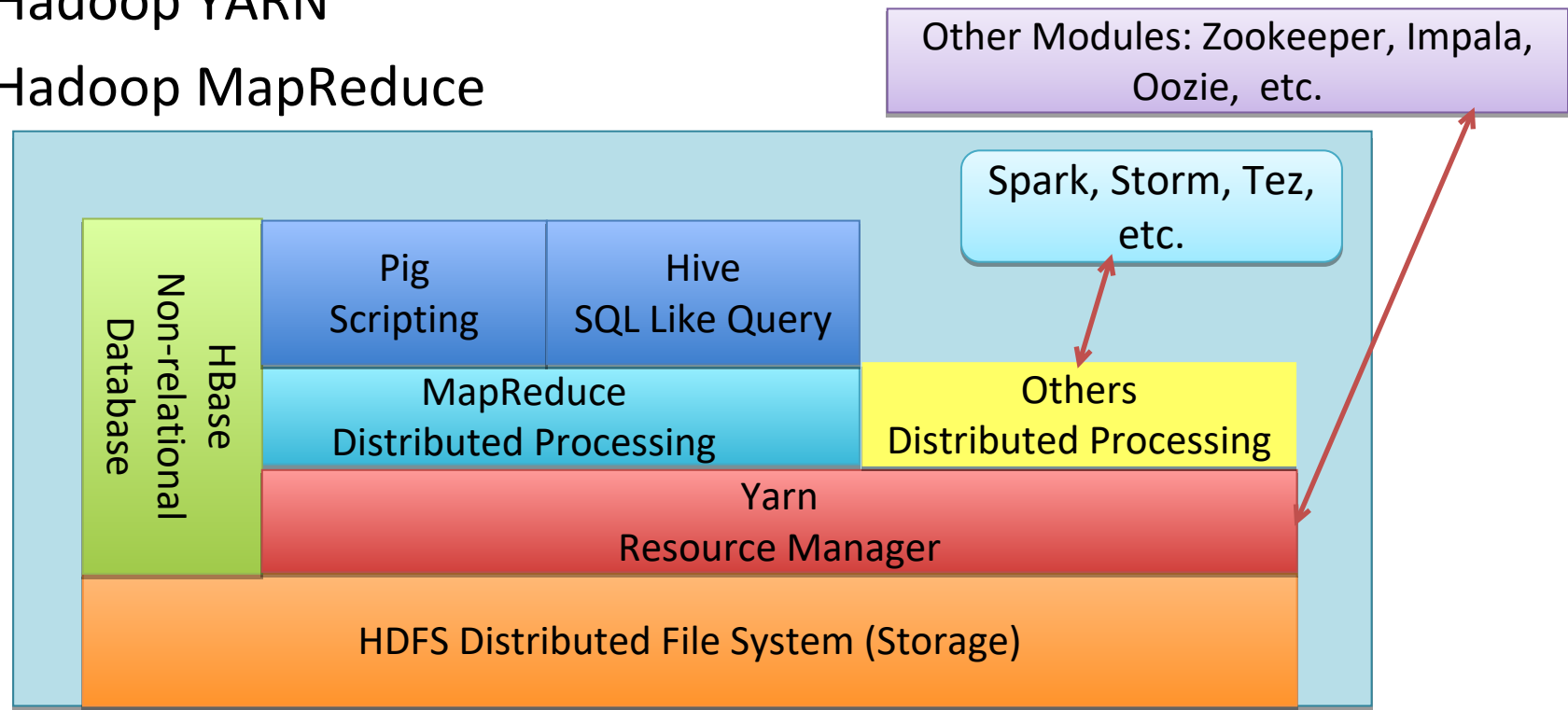
- Enable Scalability
 - on commodity hardware
- Handle Fault Tolerance
- Can Handle a Variety of Data type
 - Text, Graph, Streaming Data, Images,...
- Shared Environment
- Provides Value
 - Cost

Hadoop Ecosystem



Apache Hadoop Basic Modules

- Hadoop Common
- Hadoop Distributed File System (HDFS)
- Hadoop YARN
- Hadoop MapReduce



Hadoop HDFS

- Hadoop distributed File System (based on Google File System (GFS) paper, 2004)
 - Serves as the distributed file system for most tools in the Hadoop ecosystem
 - Scalability for large data sets
 - Reliability to cope with hardware failures
- HDFS good for:
 - Large files
 - Streaming data
- Not good for:
 - Lots of small files
 - Random access to files
 - Low latency access



Single Hadoop cluster with 5000 servers
and 250 petabytes of data

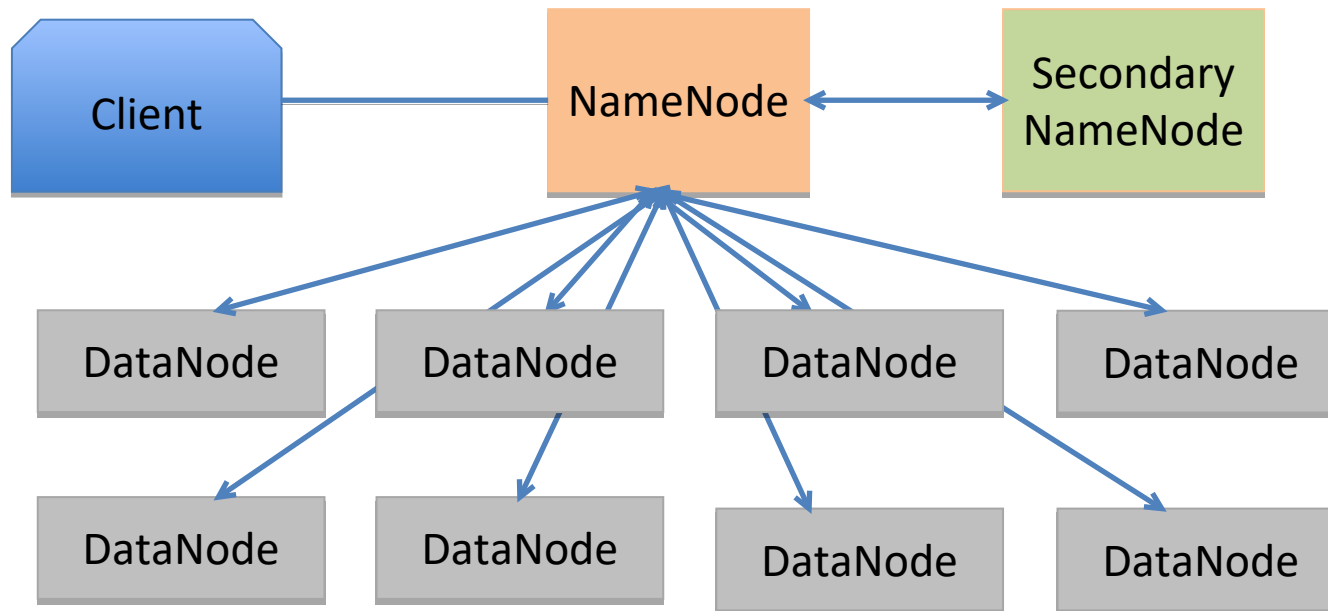
Design of Hadoop Distributed File System (HDFS)

- Master-Slave design
- Master Node
 - Single NameNode for managing metadata
- Slave Nodes
 - Multiple DataNodes for storing data
- Other
 - Secondary NameNode as a backup

HDFS Architecture

NameNode keeps the metadata, the name, location and directory

DataNode provide storage for blocks of data

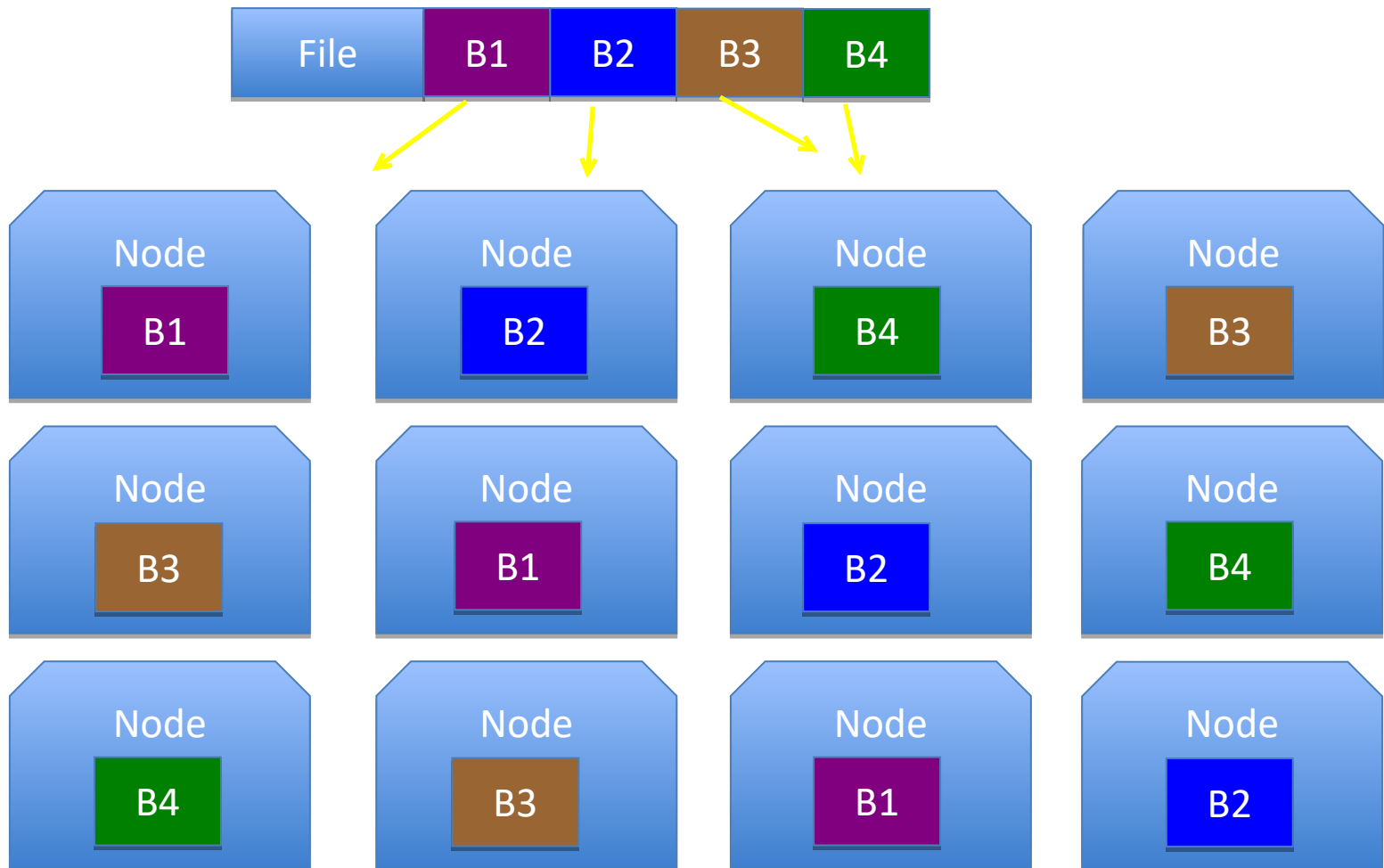


↔ Heartbeat, Cmd, Data

HDFS

What happens; if node(s) fail?

Replication of Blocks for fault tolerance



HDFS

- HDFS files are divided into blocks
 - It's the basic unit of read/write
 - Default size is 64MB, could be larger (128MB)
 - Hence makes HDFS good for storing larger files
- HDFS blocks are replicated multiple times
 - One block stored at multiple location, also at different racks (usually 3 times)
 - This makes HDFS storage fault tolerant and faster to read

Few HDFS Shell commands

Create a directory in HDFS

- `hadoop fs -mkdir /user/godil/dir1`

List the content of a directory

- `hadoop fs -ls /user/godil`

Upload and download a file in HDFS

- `hadoop fs -put /home/godil/file.txt /user/godil/datadir/`
- `hadoop fs -get /user/godil/datadir/file.txt /home/`

Look at the content of a file

- `Hadoop fs -cat /user/godil/datadir/book.txt`

Many more commands, similar to Unix

HBase

- NoSQL data store build on top of HDFS
- Based on the Google BigTable paper (2006)
- Can handle various types of data
- Stores large amount of data (TB,PB)
- Column-Oriented data store
- Big Data with random read and writes
- Horizontally scalable

HBase, not to use for

- Not good as a traditional RDBMs (Relational Database Model)
 - Transactional applications
 - Data Analytics
- Not efficient for text searching and processing

MapReduce: Simple Programming for Big Data

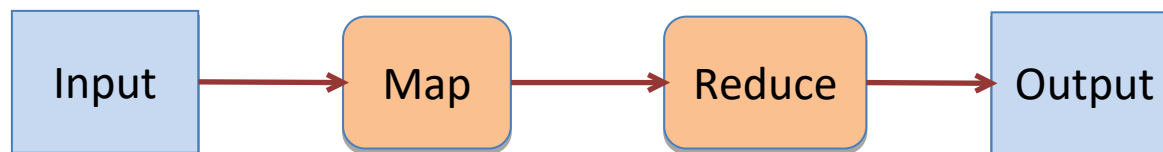
Based on Google's MR paper (2004)

- MapReduce is simple programming paradigm for the Hadoop ecosystem
- Traditional parallel programming requires expertise of different computing/systems concepts
 - examples: multithreads, synchronization mechanisms (locks, semaphores, and monitors)
 - incorrect use: can crash your program, get incorrect results, or severely impact performance
 - Usually not fault tolerant to hardware failure
- The MapReduce programming model greatly simplifies running code in parallel
 - you don't have to deal with any of above issues
 - only need to create, map and reduce functions

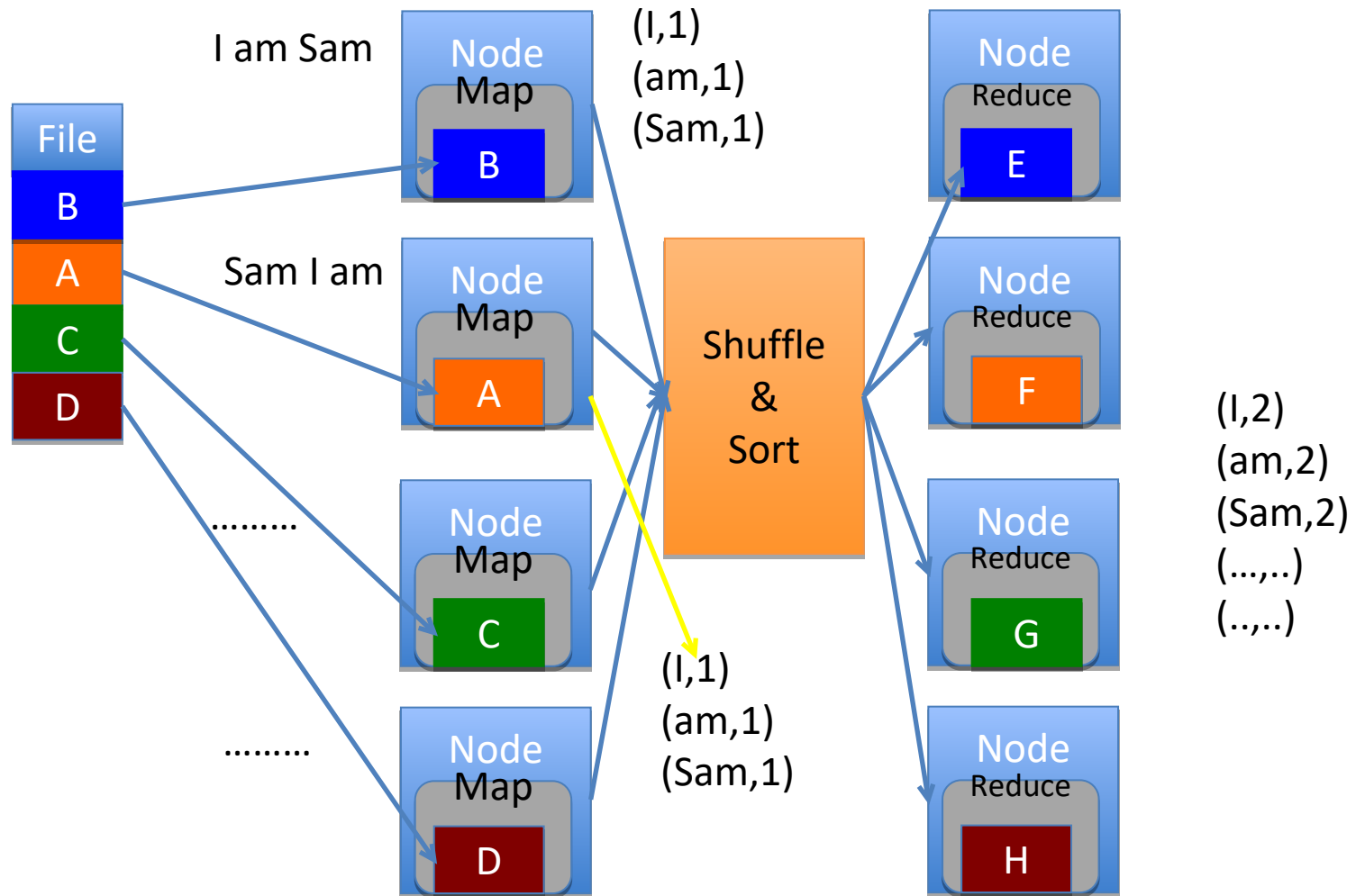
Map Reduce Paradigm

- Map and Reduce are based on functional programming

Map:	Reduce:
Apply a function to all the elements of List	Combine all the elements of list for a summary
<pre>list1=[1,2,3,4,5]; square x = x * x list2=Map square(list1) print list2 -> [1,4,9,16,25]</pre>	<pre>list1 = [1,2,3,4,5]; A = reduce (+) list1 Print A -> 15</pre>



MapReduce Word Count Example



Shortcoming of MapReduce

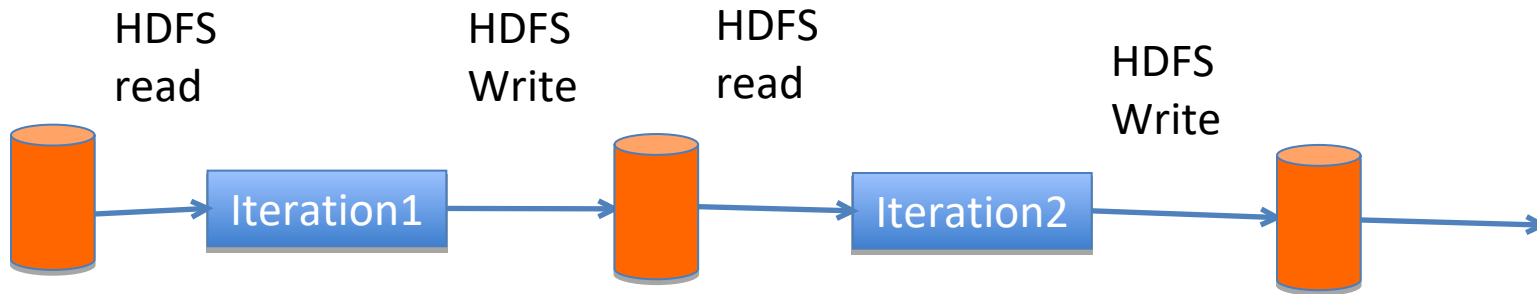
- Forces your data processing into Map and Reduce
 - Other workflows missing include join, filter, flatMap, groupByKey, union, intersection, ...
- Based on “Acyclic Data Flow” from Disk to Disk (HDFS)
- Read and write to Disk before and after Map and Reduce (stateless machine)
 - Not efficient for iterative tasks, i.e. Machine Learning
- Only Java natively supported
 - Support for others languages needed
- Only for Batch processing
 - Interactivity, streaming data

One Solution is Apache Spark

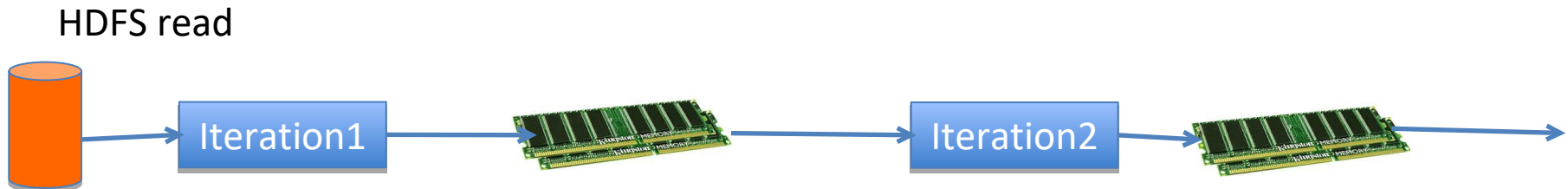
- A new general framework, which solves many of the short comings of MapReduce
- It capable of leveraging the Hadoop ecosystem, e.g. HDFS, YARN, HBase, S3, ...
- Has many other workflows, i.e. join, filter, flatMapdistinct, groupByKey, reduceByKey, sortByKey, collect, count, first...
 - (around 30 efficient distributed operations)
- In-memory caching of data (for iterative, graph, and machine learning algorithms, etc.)
- Native Scala, Java, Python, and R support
- Supports interactive shells for exploratory data analysis
- Spark API is extremely simple to use
- Developed at AMPLab UC Berkeley, now by Databricks.com

Spark Uses Memory instead of Disk

Hadoop: Use Disk for Data Sharing



Spark: In-Memory Data Sharing



Sort competition

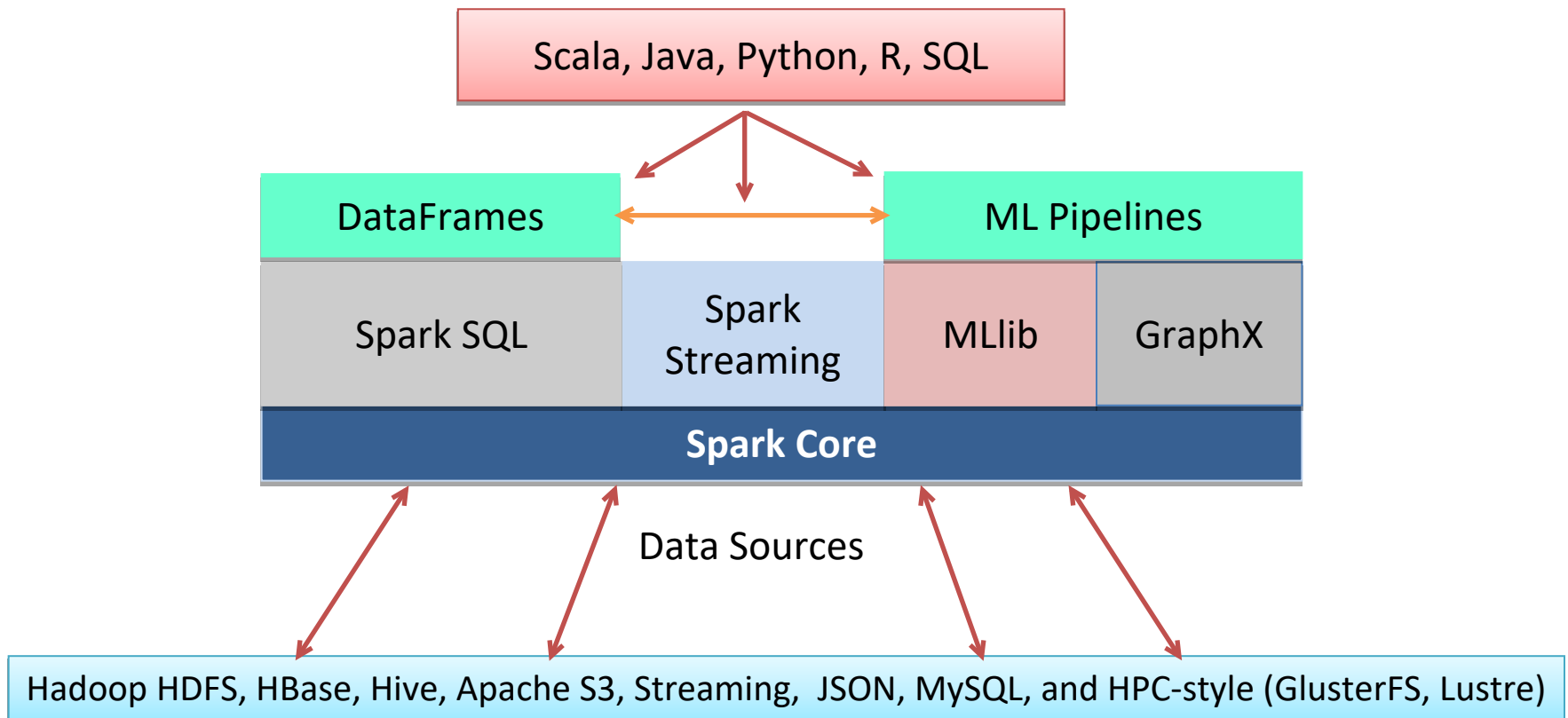
	Hadoop MR Record (2013)	Spark Record (2014)	Spark, 3x faster with 1/10 the nodes
Data Size	102.5 TB	100 TB	
Elapsed Time	72 mins	23 mins	
# Nodes	2100	206	
# Cores	50400 physical	6592 virtualized	
Cluster disk throughput	3150 GB/s (est.)	618 GB/s	
Network	dedicated data center, 10Gbps	virtualized (EC2) 10Gbps network	
Sort rate	1.42 TB/min	4.27 TB/min	
Sort rate/node	0.67 GB/min	20.7 GB/min	

Sort benchmark, Daytona Gray: sort of 100 TB of data (1 trillion records)

<http://databricks.com/blog/2014/11/05/spark-officially-sets-a-new-record-in-large-scale-sorting.html>

Apache Spark

Apache Spark supports data analysis, machine learning, graphs, streaming data, etc. It can read/write from a range of data types and allows development in multiple languages.



Resilient Distributed Datasets (RDDs)

- RDDs (Resilient Distributed Datasets) is Data Containers
- All the different processing components in Spark share the same abstraction called RDD
- As applications share the RDD abstraction, you can mix different kind of transformations to create new RDDs
- Created by parallelizing a collection or reading a file
- Fault tolerant

DataFrames & SparkSQL

- DataFrames (DFs) is one of the other distributed datasets organized in named columns
- Similar to a relational database, Python Pandas Dataframe or R's DataTables
 - Immutable once constructed
 - Track lineage
 - Enable distributed computations
- How to construct Dataframes
 - Read from file(s)
 - Transforming an existing DFs(Spark or Pandas)
 - Parallelizing a python collection list
 - Apply transformations and actions

DataFrame example

```
// Create a new DataFrame that contains “students”  
students = users.filter(users.age < 21)
```

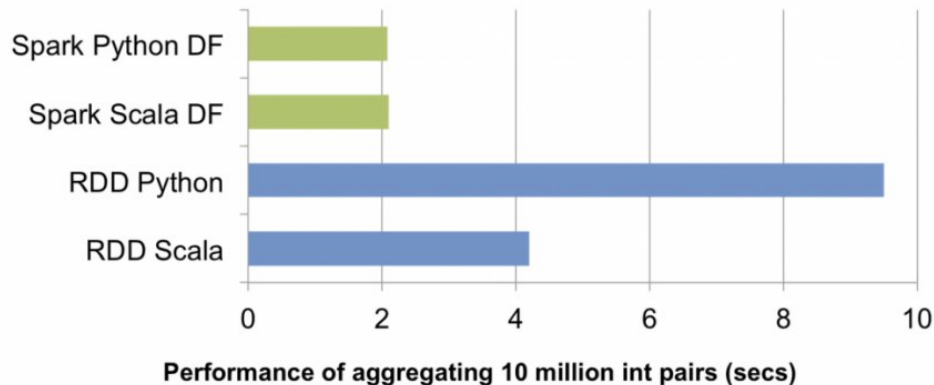
```
//Alternatively, using Pandas-like syntax  
students = users[users.age < 21]
```

```
//Count the number of students users by gender  
students.groupBy("gender").count()
```

```
// Join young students with another DataFrame called  
logs  
students.join(logs, logs.userId == users.userId,  
“left_outer”)
```

RDDs vs. DataFrames

- RDDs provide a low level interface into Spark
- DataFrames have a schema
- DataFrames are cached and optimized by Spark
- DataFrames are built on top of the RDDs and the core Spark API

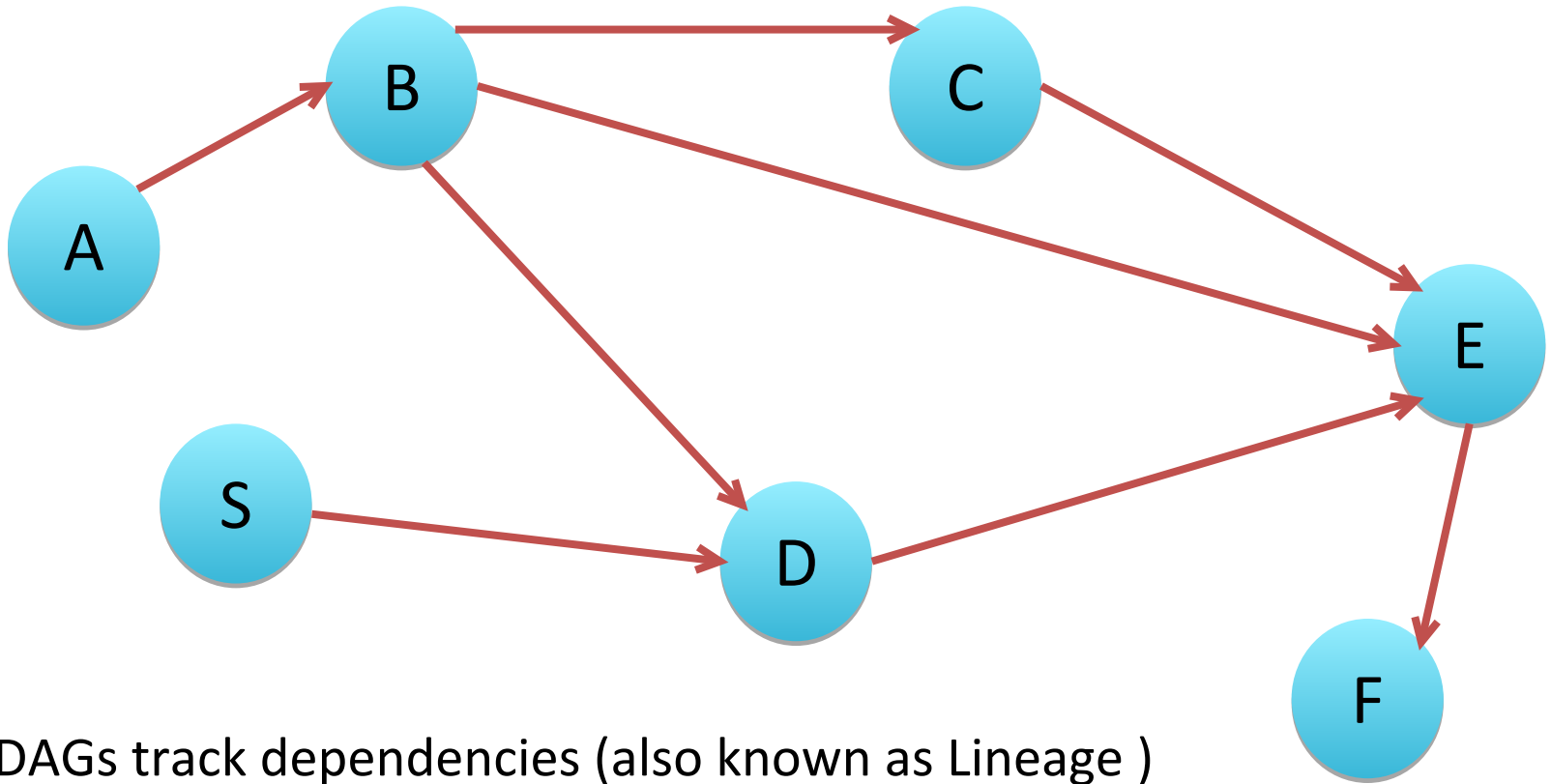


Example: performance

Spark Operations

Transformations (create a new RDD)	map filter sample groupByKey reduceByKey sortByKey intersection	flatMap union join cogroup cross mapValues reduceByKey
Actions (return results to driver program)	collect Reduce Count takeSample take lookupKey	first take takeOrdered countByKey save foreach

Directed Acyclic Graphs (DAG)



DAGs track dependencies (also known as Lineage)

- nodes are RDDs
- arrows are Transformations

Lazy evaluation (extra slide)

The standard (or **busy**) way to do this is

1. Calculate the square of each element.
2. Sum the squares.

==> This requires storing all intermediate results.

An alternative is **lazy** evaluation:

- **postpone** computing the square until result is needed.
- No need to store intermediate results.
- Scan through the data once, rather than twice.

Example

- Calculate $\sum_{i=1}^n x_i^2$

Busy Evaluation

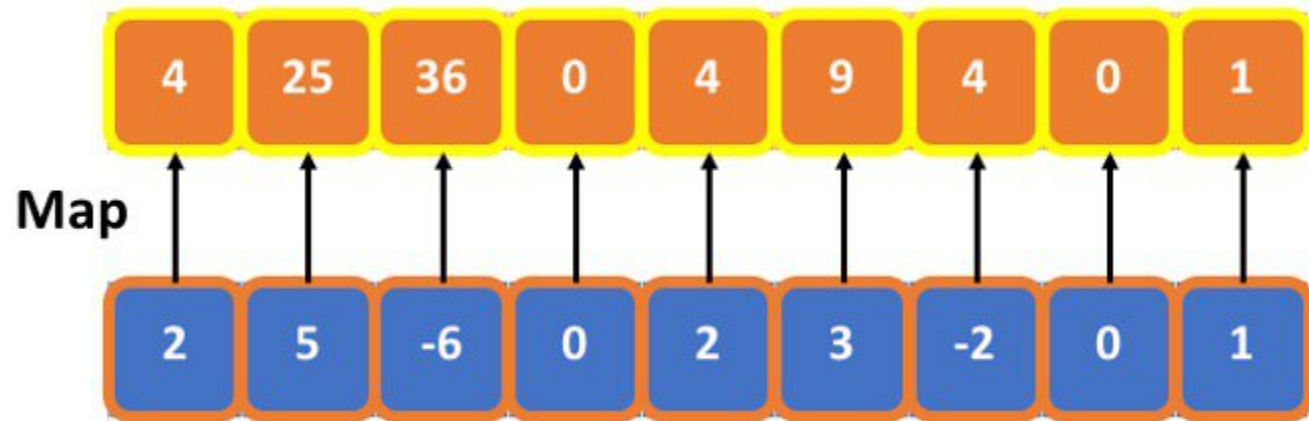
$$S = \sum_{i=1}^n x_i^2$$

2	5	-6	0	2	3	-2	0	1
---	---	----	---	---	---	----	---	---

Example

Busy Evaluation

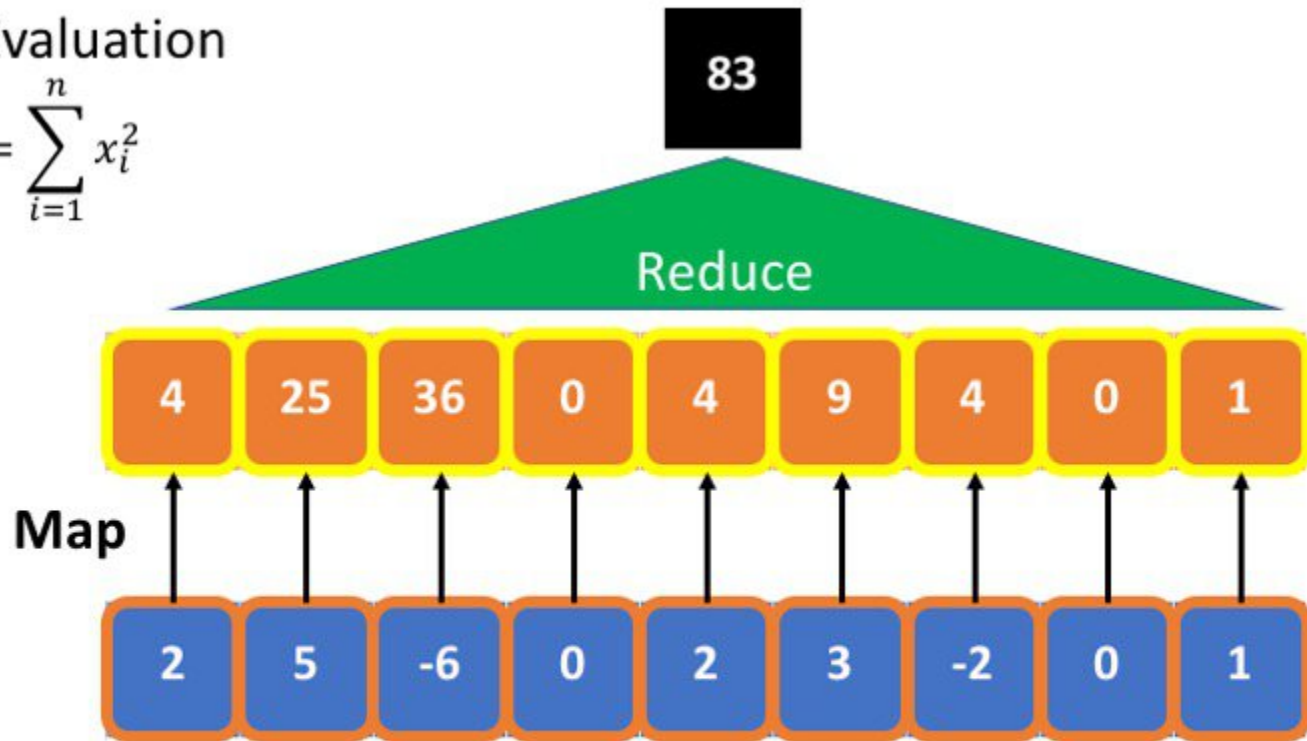
$$S = \sum_{i=1}^n x_i^2$$



Example

Busy Evaluation

$$S = \sum_{i=1}^n x_i^2$$



Example

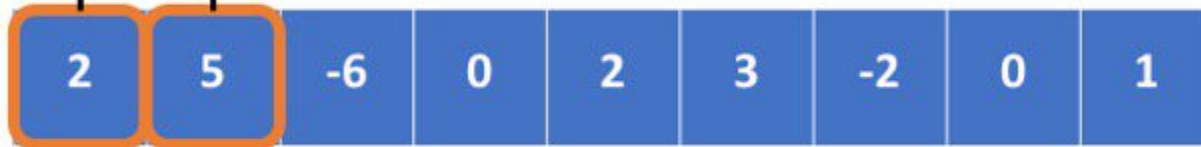
Lazy Evaluation

$$S = \sum_{i=1}^n x_i^2$$

Reduce



Map



Example

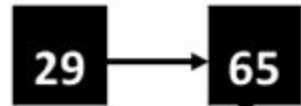
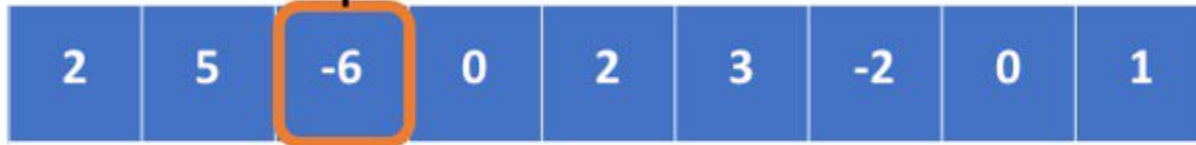
Lazy Evaluation

$$S = \sum_{i=1}^n x_i^2$$

Reduce



Map



Spark Lazy evaluation

- You have a text file of 1 GB and have created ten partitions of it.
- You also performed some transformations, and in the end, you requested to see how the first line looks.
- Spark will read the file only from the first partition and give you the results as your requested results do not require it to read the complete file.

Example

- In the first step, we created a list of 10 million numbers and made an RDD with four partitions below.

```
val data = (1 to 100000).toList  
val rdd = sc.parallelize(data,4)  
println("Number of partitions is "+rdd.getNumPartitions)
```

```
Number of partitions is 4  
data: List[Int] = List(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000)
```

Example

- perform a fundamental transformation, like adding 4 to each number
 - ➔ It only records a series of transformations in the form of RDD Lineage

```
//Adding 5 to each value in rdd
val rdd2 = rdd.map(x => x+5)
println(rdd2)
//getting rdd lineage
rdd2.toDebugString
```

```
MapPartitionsRDD[15] at map at command-1376195624900326:2
rdd2: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[15] at map at command-1376195624900326:2
res1: String =
(4) MapPartitionsRDD[15] at map at command-1376195624900326:2 []
|   ParallelCollectionRDD[14] at parallelize at command-1376195624900328:2 []
```

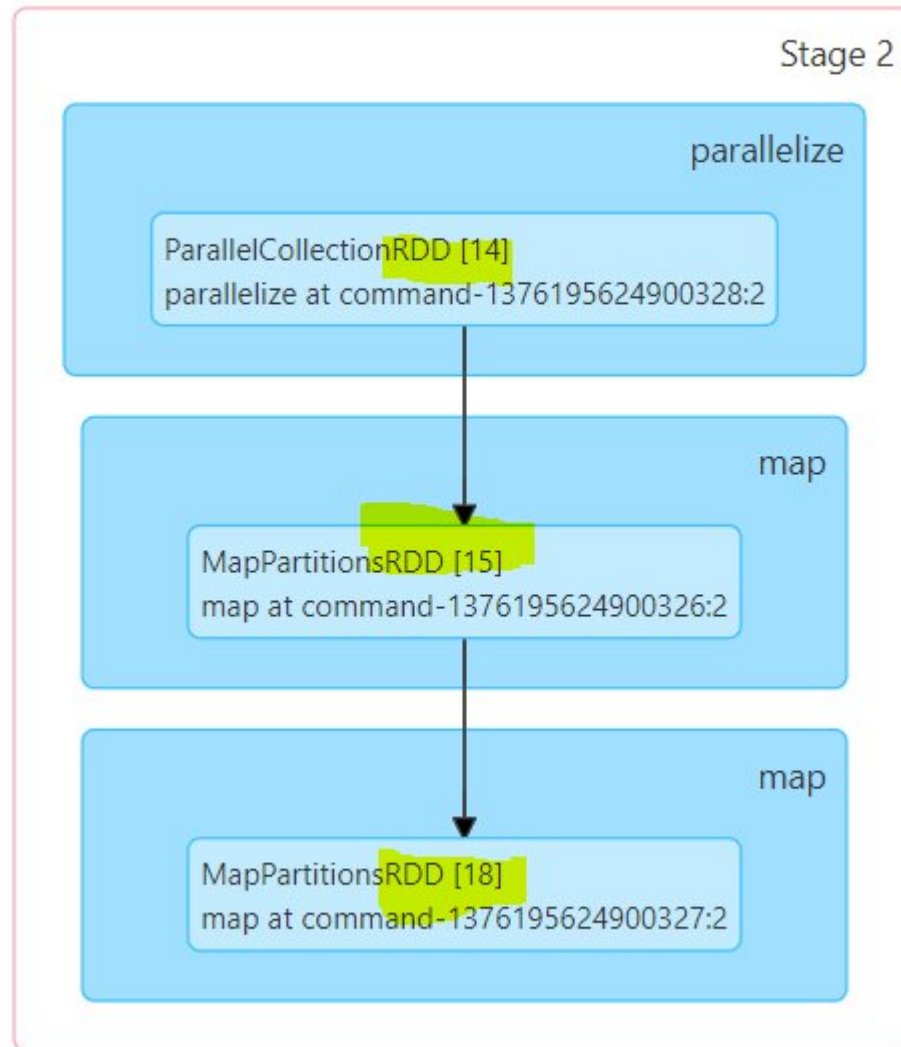
Example

- adding 4 to each number

```
//Adding 5 to each value in rdd  
val rdd3 = rdd2.map(x => x+20)  
println(rdd3)  
//getting rdd lineage  
rdd3.toDebugString  
rdd3.collect
```

```
MapPartitionsRDD[18] at map at command-1376195624900327:2  
rdd3: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[18] at map at command-1376195624900327:2  
res4: String =  
(4) MapPartitionsRDD[18] at map at command-1376195624900327:2 []  
| MapPartitionsRDD[15] at map at command-1376195624900326:2 []  
| ParallelCollectionRDD[14] at parallelize at command-1376195624900328:2 []
```

▼ DAG Visualization



Example 2

```
import org.apache.spark.sql.functions._  
val df1 = (1 to 100000).toList.toDF("col1")  
//scenario 1  
println("scenario 1")  
df1.withColumn("col2",lit(2)).explain(true);  
//scenario 2  
println("scenario 2")  
df1.withColumn("col2",lit(2)).drop("col2").explain(true);
```

Example 2

```
import org.apache.spark.sql.functions._
val df1 = (1 to 1000).rdd.map(_ * 2).toDF()
//scenario 1
println("scenario 1")
df1.withColumn("col2", col("col1") * 2)
//scenario 2
println("scenario 2")
df1.withColumn("col2", col("col1") * 2)
```

```
scenario 1
== Parsed Logical Plan ==
Project [col1#236, 2 AS col2#238]
+- Project [value#233 AS col1#236]
   +- LocalRelation [value#233]

== Analyzed Logical Plan ==
col1: int, col2: int
Project [col1#236, 2 AS col2#238]
+- Project [value#233 AS col1#236]
   +- LocalRelation [value#233]

== Optimized Logical Plan ==
LocalRelation [col1#236, col2#238]

== Physical Plan ==
LocalTableScan [col1#236, col2#238]
```

Example 2

```
import org.apache.spark.sql
val df1 = (1 to 100000).toList
//scenario 1
println("scenario 1")
df1.withColumn("col2",lit(2))
//scenario 2
println("scenario 2")
df1.withColumn("col2",lit(2))
```

```
scenario 2
== Parsed Logical Plan ==
Project [col1#236]
+- Project [col1#236, 2 AS col2#241]
   +- Project [value#233 AS col1#236]
      +- LocalRelation [value#233]

== Analyzed Logical Plan ==
col1: int
Project [col1#236]
+- Project [col1#236, 2 AS col2#241]
   +- Project [value#233 AS col1#236]
      +- LocalRelation [value#233]

== Optimized Logical Plan ==
LocalRelation [col1#236]

== Physical Plan ==
LocalTableScan [col1#236]
```

there is no task of creation and of "col2"

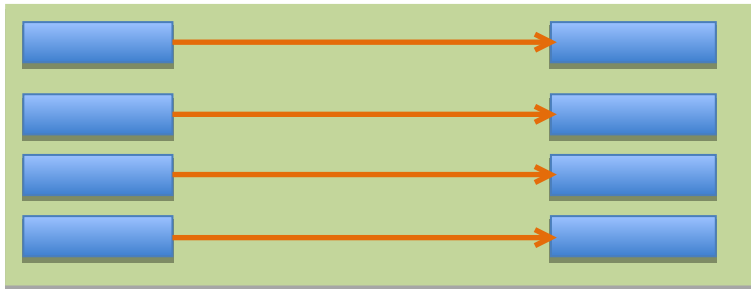
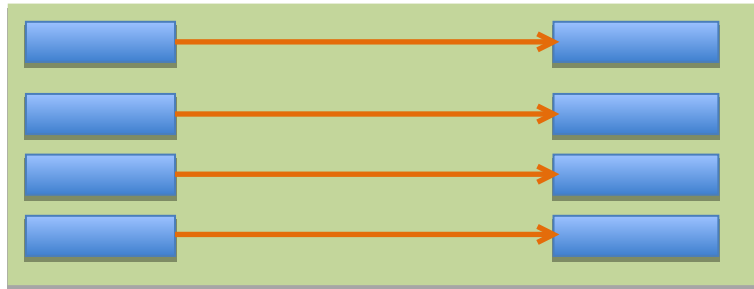
Advantages of Spark Lazy Evaluation

- Users can divide the entire work into smaller operations for easy readability and management
- But, Spark internally groups the transformations reducing the number of passes on data
- the action is triggered only when data is required

→ only the necessary computation is done by Spark, which increases Speed.

Narrow Vs. Wide transformation

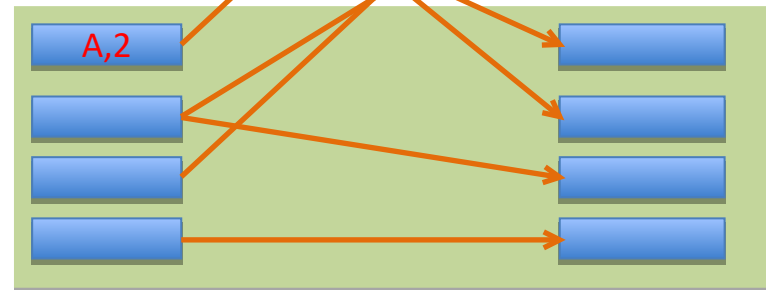
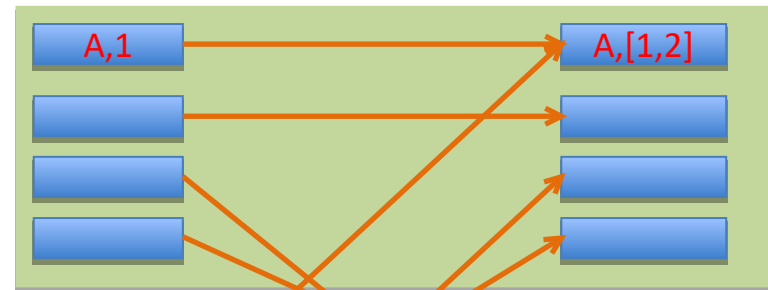
Narrow



Map

Vs.

Wide

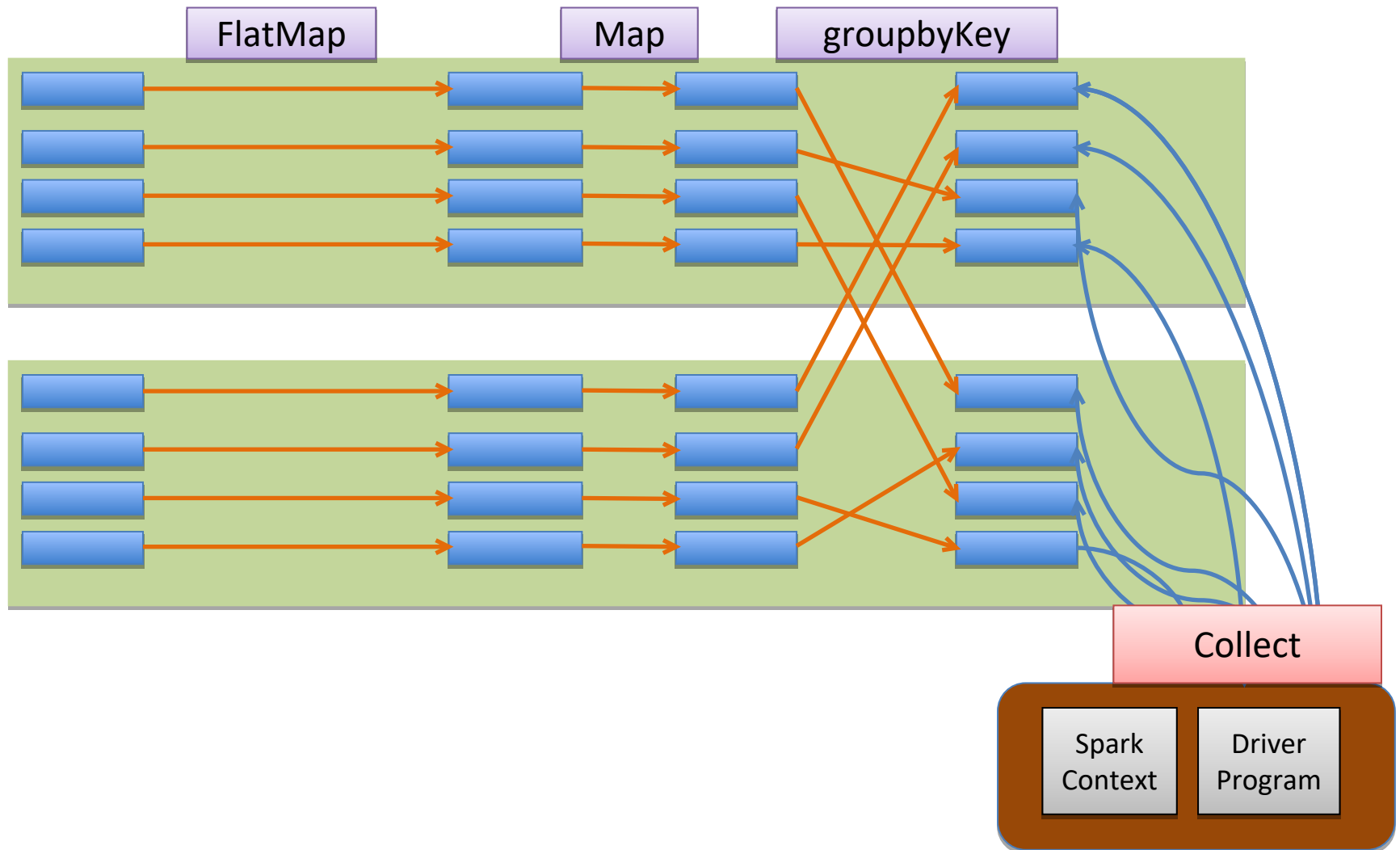


groupByKey

Actions

- What is an action
 - The final stage of the workflow
 - Triggers the execution of the DAG
 - Returns the results to the driver
 - Or writes the data to HDFS or to a file

Spark Workflow



Python RDD API Examples

- Word count

```
text_file = sc.textFile("hdfs://usr/godil/text/book.txt")
counts = text_file.flatMap(lambda line: line.split(" ")) \
    .map(lambda word: (word, 1)) \
    .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("hdfs://usr/godil/output/wordCount.txt")
```

- Logistic Regression

```
# Every record of this DataFrame contains the label and
# features represented by a vector.
df = sqlContext.createDataFrame(data, ["label", "features"])
# Set parameters for the algorithm.
# Here, we limit the number of iterations to 10.
lr = LogisticRegression(maxIter=10)
# Fit the model to the data.
model = lr.fit(df)
# Given a dataset, predict each point's label, and show the results.
model.transform(df).show()
```

RDD Persistence and Removal

- RDD Persistence
 - `RDD.persist()`
 - Storage level:
 - `MEMORY_ONLY`, `MEMORY_AND_DISK`, `MEMORY_ONLY_SER`, `DISK_ONLY`,.....
- RDD Removal
 - `RDD.unpersist()`

Broadcast Variables and Accumulators (Shared Variables)

- Broadcast variables allow the programmer to keep a read-only variable cached on each node, rather than sending a copy of it with tasks
- Accumulators are variables that are only “added” to through an associative operation and can be efficiently supported in parallel

```
>broadcastV1 = sc.broadcast([1, 2, 3,4,5,6])
```

```
>broadcastV1.value
```

```
[1,2,3,4,5,6]
```

```
accum = sc.accumulator(0)
```

```
accum.add(x)
```

```
accum.value
```

Spark's Main Use Cases

- Streaming Data
- Machine Learning
- Interactive Analysis
- Data Warehousing
- Batch Processing
- Exploratory Data Analysis
- Graph Data Analysis
- Spatial (GIS) Data Analysis
- And many more

My Spark Use Cases

- Fingerprint Matching
 - Developed a Spark based fingerprint minutia detection and fingerprint matching code
- Twitter Sentiment Analysis
 - Developed a Spark based Sentiment Analysis code for a Twitter dataset

Spark in the Real World (I)

- Uber – the online taxi company gathers terabytes of event data from its mobile users every day.
 - By using Kafka, Spark Streaming, and HDFS, to build a continuous ETL pipeline
 - Convert raw unstructured event data into structured data as it is collected
 - Uses it further for more complex analytics and optimization of operations
- Pinterest – Uses a Spark ETL pipeline
 - Leverages Spark Streaming to gain immediate insight into how users all over the world are engaging with Pins—in real time.
 - Can make more relevant recommendations as people navigate the site
 - Recommends related Pins
 - Determine which products to buy, or destinations to visit

Spark in the Real World (II)

Here are Few other Real World Use Cases:

- Conviva – 4 million video feeds per month
 - This streaming video company is second only to YouTube.
 - Uses Spark to reduce customer churn by optimizing video streams and managing live video traffic
 - Maintains a consistently smooth, high quality viewing experience.
- Capital One – is using Spark and data science algorithms to understand customers in a better way.
 - Developing next generation of financial products and services
 - Find attributes and patterns of increased probability for fraud
- Netflix – leveraging Spark for insights of user viewing habits and then recommends movies to them.
 - User data is also used for content creation

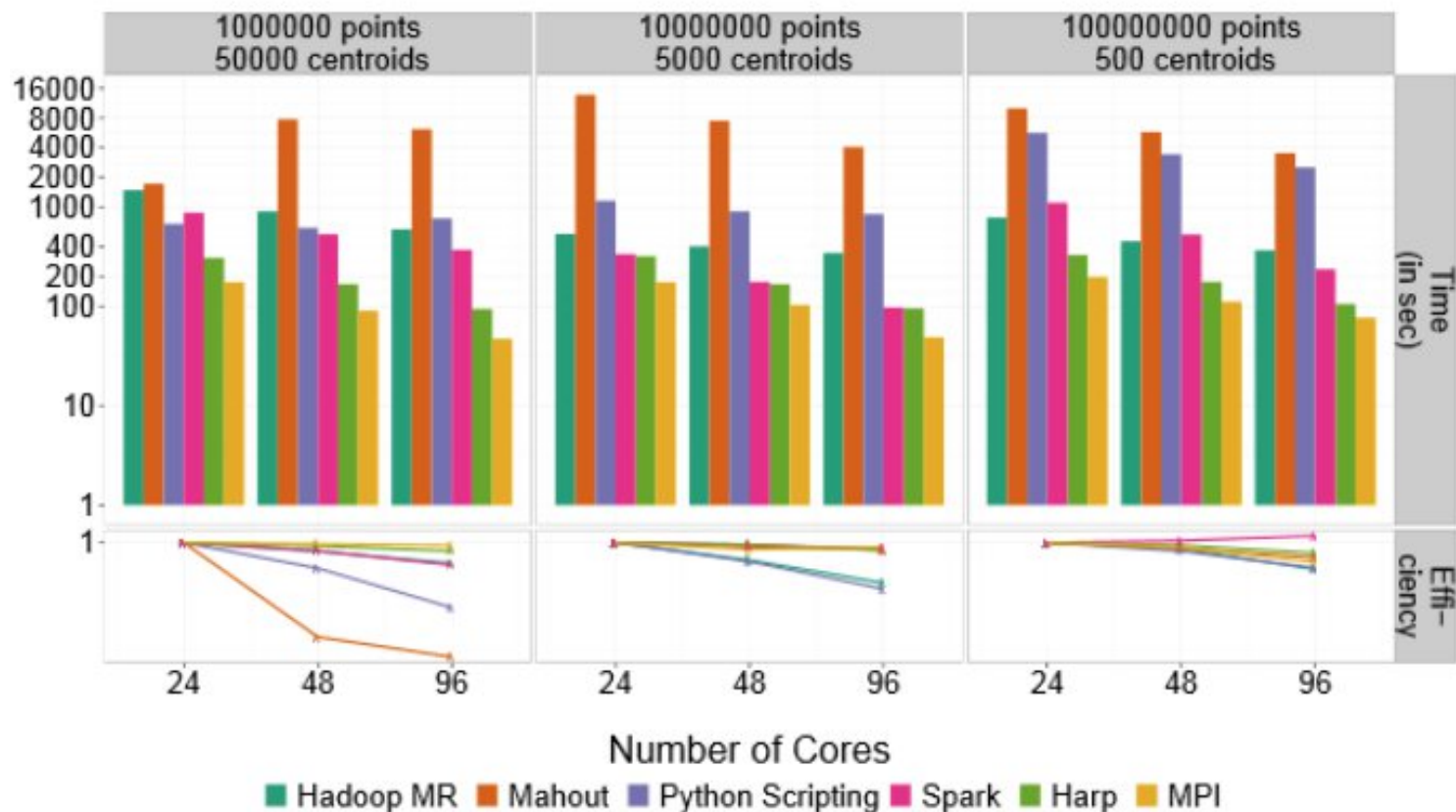
Spark: when not to use

- Even though Spark is versatile, that doesn't mean Spark's in-memory capabilities are the best fit for all use cases:
 - For many simple use cases Apache MapReduce and Hive might be a more appropriate choice
 - Spark was not designed as a multi-user environment
 - Spark users are required to know that memory they have is sufficient for a dataset
 - Adding more users adds complications, since the users will have to coordinate memory usage to run code

HPC and Big Data Convergence

- Clouds and supercomputers are collections of computers networked together in a datacenter
- Clouds have different networking, I/O, CPU and cost trade-offs than supercomputers
- Cloud workloads are data oriented vs. computation oriented and are less closely coupled than supercomputers
- Principles of parallel computing same on both
- Apache Hadoop and Spark vs. Open MPI

HPC and Big Data K-Means example



MPI definitely outpaces Hadoop, but can be boosted using a hybrid approach of other technologies that blend HPC and big data, including Spark and HARP. Dr. Geoffrey Fox, Indiana University. (<http://arxiv.org/pdf/1403.1528.pdf>)

Conclusion

- Hadoop (HDFS, MapReduce)
 - Provides an easy solution for processing of Big Data
 - Brings a paradigm shift in programming distributed system
- Spark
 - Has extended MapReduce for in memory computations
 - for streaming, interactive, iterative and machine learning tasks
- Changing the World
 - Made data processing cheaper and more efficient and scalable
 - Is the foundation of many other tools and software

Backup

MapReduce vs. Spark for Large Scale Data Analysis

- MapReduce and Spark are two very popular open source cluster computing frameworks for large scale data analytics
- These frameworks hide the complexity of task parallelism and fault-tolerance, by exposing a simple programming API to users

Tasks	Word Count	sort	K-means	Page-Rank
MapReduce				
Spark				